TWO-DAY TECHNOLOGY RESIDENTIAL COURSE

From Idea to Application

What Every CA Should Know Before Building Software

Understanding technology choices before you write a single line of code.

PART 1

What Is Code, Really?

You already think like a programmer. You just haven't called it that.

0 – 5 MINUTES

Why Should a CA Understand Coding at All?

- You are not being trained to become a developer in two days.
- You are being trained to make good decisions about tools that will run your firm's work and touch client data.
- Vibe coding tools let you describe an idea in plain language and get a working prototype in minutes.
- But the tool will make technology decisions for you silently — front end, database, hosting — unless you know what to ask for.
- This hour gives you the vocabulary and judgement to stay in control of that conversation.

5 – 15 MINUTES

What Is Code and Coding?

Code is a set of instructions given to a computer to perform a task.

Coding is the process of writing those instructions clearly, logically, and repeatedly.

A good CA already thinks like a programmer when designing a checklist, audit procedure, reconciliation logic, or compliance workflow. Coding only converts that thinking into a tool.

So coding is not magic — it is structured thinking converted into machine-readable instructions.

5 – 15 MINUTES

From Audit Program to Code Logic

Your audit program says:

- Check whether invoice is available
- Match invoice with purchase register
- Verify GSTIN
- Match ITC with GSTR-2B
- Report mismatch

The same logic in code becomes:

- Take invoice data
- Read GSTIN
- Compare with vendor master
- Compare tax amount with GSTR-2B
- Show exceptions

5 – 15 MINUTES

Three Levels of Getting Work Done

Manual process

A person does the work using judgement and documents.

Automated process

Software performs repeated steps on its own.

Assisted process

Software helps the person decide, draft, extract, summarise, or validate.

PART 2

Components of an Application

Every application you build this weekend has the same anatomy.

15 – 30 MINUTES

The Three Basic Components

Front End

The reception desk of the application.

- Login screen
- Dashboard
- Data entry forms
- Upload buttons
- Reports & approval screens

Back End

The office staff and processing team.

- Who can log in
- What happens after upload
- Approval routing
- Draft generation logic
- Report calculations

Database

The record room of the application.

- Client & employee master
- GST notices, invoices
- Tasks & approval history
- User roles

15 – 30 MINUTES

Front End: The Reception Desk

- What the user sees and directly interacts with.
- Examples: login screen, dashboard, data-entry forms, upload buttons, reports, approval screens.
- Common technologies: HTML, CSS, JavaScript, React, Angular, Vue, Flutter, Electron front end.
- For your RRC project: this is what your group will spend the most visible time on, but it is the smallest part of what makes the tool trustworthy.

15 – 30 MINUTES

Back End: The Office Staff Behind the Desk

- The logic layer — decides what actually happens.
- Examples: who can log in, what happens after invoice upload, how approval moves from employee to manager, how a GST notice reply draft is generated, how data validation and report calculations happen.
- Common technologies: Python, Node.js, Java, .NET, PHP, Go.
- This is where your project's actual business rules live — the audit logic, the approval workflow, the reconciliation.

15 – 30 MINUTES

Database: The Record Room

- Where information is stored, permanently.
- Examples: client master, employee master, GST notices, invoice details, tasks, approval history, user roles.
- Common databases: SQLite, PostgreSQL, MySQL, SQL Server, MongoDB, Supabase database.
- Ask early: what happens to this data if the laptop is lost, or the browser tab is closed? That question alone tells you whether you need a real database.

15 – 30 MINUTES

The Hidden Components Nobody Demos

Hidden area	Why it matters
Authentication	Who can log in
Authorization	What each user is allowed to do
Hosting	Where the application actually runs
Security	How data is protected
Backup	How data is recovered if something fails
Version updates	How fixes and features reach users
Audit trail	Who changed what, and when
File storage	Where uploaded PDFs, Excel files, images live
Integration	Connection with GST portal, Tally, email, payment gateway
Support model	Who helps users when something breaks

15 – 30 MINUTES

When a Tool Becomes a Product

Your own Excel tool

- Used by you, on your machine
- Password protection may be enough
- No client data leaving your desk
- Low stakes if something breaks

A client-facing GST notice application

- Login required
- Role-based access control
- Document security
- Data backup
- Activity log
- Version control, hosting, user support

This is where a tool becomes a product.

PART 3

Choosing the Right Platform

Excel VBA vs HTML vs Desktop vs Cloud — matched to your problem, not your excitement.

30 – 45 MINUTES

When to Use Excel VBA

Use it when

- The problem is small and internal
- Data is already in Excel
- Used by one person or a small team
- Logic is repetitive
- Output is a report, reconciliation, checklist, or format
- e.g. GST ITC reconciliation, TDS working automation, depreciation calculator, audit sampling tool

Avoid it when

- Many users need simultaneous access
- Strong data security is required
- Web or mobile access is required
- Long-term productization is planned
- Version control across users becomes difficult

30 – 45 MINUTES

When to Use a Simple HTML Application

Use it when

- You need a form-like interface
- No heavy database is required
- A lightweight demo is the goal
- It's mostly a calculator, checklist, or static workflow
- The app can run entirely in a browser
- e.g. GST interest calculator, due date finder, eligibility calculator, compliance checklist

Avoid it when

- Data must be saved permanently
- Multiple users need login
- Approval workflow is required
- Confidential data is involved
- Files must be uploaded and processed

30 – 45 MINUTES

When to Use a Desktop Application

Use it when

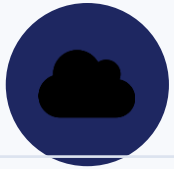
- The user works mostly on one computer
- Data should remain inside the office system
- Internet may not be reliable
- Files are large
- Local processing matters
- Integration with local files, printers, scanners, DSC, or Tally is needed

Avoid it when

- Users are spread across locations
- Real-time collaboration is needed
- A central database is needed
- Frequent updates are required
- Users need mobile or browser access

30 – 45 MINUTES

When to Use a Cloud Application



Use it when

- Multiple users need access
- Users work from different locations
- Role-based access is needed for partners, managers, staff, clients
- Data should be centralised
- Workflow and approval tracking is required
- The product may eventually be sold to others

Avoid it when

- Data cannot leave local premises
- Internet dependency is unacceptable
- Budget is very limited
- The problem is small enough for Excel or desktop

30 – 45 MINUTES

Simple Decision Matrix

Requirement	Best choice
Personal automation	Excel VBA
Calculator or checklist	HTML + JavaScript
Local file-heavy utility	Desktop app
Multi-user workflow	Cloud app
Mobile access	Cloud or mobile app
Client-facing portal	Cloud app
Tally / local system integration	Desktop or hybrid
Quick prototype	Lovable, Replit, Bolt, Cursor
Long-term SaaS product	Cloud app with proper architecture

PART 4

Languages & Platforms

Names your vibe coding tool will mention. Not a syllabus — a map.

45 – 55 MINUTES

Python

The default choice for data, automation, and AI-assisted tools.

Used for

- Data processing & automation
- AI and machine learning
- PDF and Excel processing
- Scripting, backend APIs

Strengths & limits

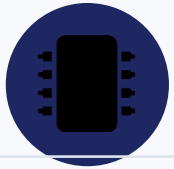
- + Easy to learn, reads clearly
- + Very large library ecosystem
- + Strong for data and AI work
- – Desktop packaging can be tricky
- – Less preferred for heavy enterprise front ends

Good CA use cases

- GST data reconciliation
- Excel automation
- PDF extraction
- AI-assisted document review
- Data analytics

45 – 55 MINUTES

C



Like learning how the engine works — useful, but not the first choice for you.

Used for

- System programming
- Operating systems
- Embedded systems
- High-performance software

Strengths & limits

- + Very fast, low-level control
- + Used where hardware control matters
- − Steep learning curve
- − Not designed for rapid business-app development

Good CA use cases

- Not usually the first choice for CA firm applications
- Useful background knowledge only

45 – 55 MINUTES

.NET

Microsoft's mature framework for Windows and enterprise business tools.

Used for

- Windows desktop applications
- Enterprise applications & APIs
- Internal business tools
- Microsoft ecosystem apps

Strengths & limits

- + Strong for Windows environments
- + Good performance and tooling
- + Mature, well-supported framework

Good CA use cases

- Desktop audit tool
- PDF utility
- Local GST filing utility
- Internal office software
- Apps needing Windows integration

45 – 55 MINUTES

Electron

Desktop apps built with web technology — the engine behind VS Code and Slack.

Used for

- Desktop apps built using web technologies
- Apps needing the same look on Windows, Mac, Linux
- Examples in the market: VS Code, Slack desktop, Discord

Strengths & limits

- + Uses HTML, CSS, JavaScript
- + One codebase across operating systems
- + Good UI flexibility
- – App size can be large
- – Higher memory usage
- – Local system integration needs care

Good CA use cases

- Desktop audit documentation tool
- Offline-first working paper tool
- Client file manager
- Local app with a modern UI

45 – 55 MINUTES

React

Generally the face of the application — not the whole application.

Used for

- Web application user interfaces
- Dashboards, forms, portals
- SaaS product front ends

Strengths & limits

- + Flexible, large community
- + Works well for modern applications
- + Good for reusable components
- − Needs a backend and database to be a full application

Good CA use cases

- Compliance dashboard
- Client portal
- Audit workflow screen
- MIS dashboard
- Document collection portal

45 – 55 MINUTES

Java

Stable and scalable — the backbone of many banking and enterprise systems.

Used for

- Enterprise applications
- Banking systems
- Large-scale backend systems
- Android development

Strengths & limits

- + Stable and scalable
- + Strong enterprise adoption
- − More verbose to write
- − Can feel heavy for small prototypes

Good CA use cases

- Large enterprise compliance systems
- Banking integrations
- Large workflow systems
- High-volume backend services

45 – 55 MINUTES

JavaScript / Node.js

One language for both front end and back end — why many vibe coding tools default here.

Used for

- Web front end
- Back end using Node.js
- APIs
- Real-time applications

Strengths & limits

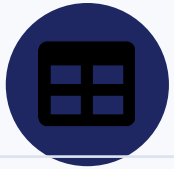
- + Same language on front end and back end
- + Huge ecosystem
- + Works well with React, Electron, Next.js

Good CA use cases

- SaaS applications
- Client portals
- Workflow apps
- Dashboards

45 – 55 MINUTES

SQL — Probably More Useful to You Than Any Language



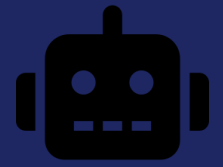
SQL is the language you use to talk to a database directly.

- Show all pending tasks
- Find invoices above ₹5 lakh
- List clients whose GST return is pending
- Show overdue receivables
- Summarise expenses by ledger

For business applications, SQL is often more useful to a CA than learning a complex programming language.

PART 5

Vibe Coding & Your RRC Project



From this hour to what you'll actually do tomorrow morning.

55 – 60 MINUTES

What Vibe Coding Actually Is

- Vibe coding means using AI tools to generate software by describing what you want in natural language.
- AI can generate code — and AI can make mistakes.
- You must still understand the workflow you're asking for.
- You must still test what the tool produces.
- You must still know what data is being stored, and where.
- You must still check security and access rights before anyone else uses it.
- Never blindly deploy confidential client data into a tool you haven't verified.

55 – 60 MINUTES

Five Questions Before You Open the Tool

1	**Who will use it?**	Only me → Excel/desktop • My team → desktop/cloud • Clients too → cloud
2	**Where is the data?**	In Excel → VBA/Python • In PDFs → Python/cloud • In Tally → desktop/hybrid
3	**Is collaboration required?**	No → Excel, HTML, or desktop • Yes → cloud
4	**Is internet always available?**	No → desktop • Yes → cloud
5	**Is this a product or a tool?**	Tool → Excel, HTML, desktop • Product → cloud or packaged desktop

55 – 60 MINUTES

Mapping This to Your Ten RRC Projects

Project	Suggested platform
GST Notice Response Assistant	Cloud or desktop
Audit Planning Assistant	Cloud
Expense Reimbursement Workflow	Cloud
Statutory Compliance Calendar	Cloud
Internal Audit Observation Tracker	Cloud
Invoice Data Extraction & Review Tool	Desktop or cloud
Client Document Collection Portal	Cloud
Cash Flow Projection App	Cloud
Staff Timesheet & Task Tracker	Cloud
Management MIS Dashboard Builder	Cloud or desktop

CLOSING THOUGHT

As Chartered Accountants, we understand process, risk, evidence, compliance, and controls — the foundations of good software.

We need not become full-time programmers to build useful applications. But we must understand enough technology to ask the right questions, choose the right platform, test the right risks, and build tools that are useful beyond a demo.

One-line takeaway

Do not start with the language. Start with the user, workflow, data, security, and scale. The technology choice should follow the problem, not the other way around.